

Материалы кафедры ММП факультета ВМК МГУ. Практикум по программированию на языке Python.

Занятие 03: Numpy и матрично- векторные вычисления

Занятие провел: Захаров Александр (tg:
@krachmath)

Материалы составили: Захаров Александр
(tg: @krachmath), Булкин Антон (tg:
@pochti_chekhov), Воробьев Сергей (tg:
@serv01), Находнов Максим
(nakhodnov17@gmail.com)

Москва, Осенний семестр 2026

Источники:

[NumPy](#) – документация по NumPy

[SciPy](#) – документация по SciPy

[NetworkX](#) – документация по NetworkX

О чём можно узнать из этого ноутбука:

- операции при работе с массивами
- многомерные массивы
- изменение размеров массивов
- broadcasting
- продвинутая индексация
- view и копирование

- свёртка
- разные прикладные задачи

```
In [1]: import numpy as np

from scipy import linalg
from scipy import optimize
from scipy.sparse import coo_matrix
from scipy import stats

import networkx as nx

import matplotlib.pyplot as plt

import time
```

Представление матрицы в Python

Простейший вариант – список списков:

```
In [2]: A = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
]

B = [
    [1, 0, 0],
    [0, 2, 0],
    [0, 0, 3],
]

def print_matrix(A):
    for row in A:
        print(row)
    print()

print_matrix(A)
print_matrix(B)
```

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

```
[1, 0, 0]
[0, 2, 0]
[0, 0, 3]
```

Задача 1. Задайте матрицу `C` размером 9 на 10, в которой ij -ый элемент будет равен $10i + j$. Не используйте библиотеку `numpy` !

```
In [ ]: ### your code here
```

Проверьте себя:

```
In [4]: assert len(C) == 9
assert len(C[0]) == 10
assert C[8][3] == 83

print_matrix(C)
```

```
[0 1 2 3 4 5 6 7 8 9]
[10 11 12 13 14 15 16 17 18 19]
[20 21 22 23 24 25 26 27 28 29]
[30 31 32 33 34 35 36 37 38 39]
[40 41 42 43 44 45 46 47 48 49]
[50 51 52 53 54 55 56 57 58 59]
[60 61 62 63 64 65 66 67 68 69]
[70 71 72 73 74 75 76 77 78 79]
[80 81 82 83 84 85 86 87 88 89]
```

Одна из проблем списков – отсутствие поэлементных и матричных операций.

Другой недостаток – работа со списками не позволяет использовать векторные инструкции в процессоре, которые на порядки ускоряют матричные вычисления

Попробуем самостоятельно реализовать несколько методов для матричных вычислений, работающих с представлением матриц в виде списков.

Для оценки производительности будем использовать *декооратор* `timed` :

```
In [5]: def timed(method):
    def __timed(*args, **kw):
        time_start = time.time()
        result = method(*args, **kw)
        time_end = time.time()

        print('{ } {:.3f} ms\n'.format(method.__name__,
                                         (time_end - time_start) * 1000))

        return result

    return __timed
```

Базовые операции

Сложение

```
In [6]: @timed
def matrix_add(A, B):
    # skip correctness of dimensions check
```

```

n, m = len(A), len(A[0])
C = [[0.0 for _ in range(m)] for _ in range(n)]

for i in range(n):
    for j in range(m):
        C[i][j] = A[i][j] + B[i][j]
    return C

print_matrix(matrix_add(A, B))

```

matrix_add 0.006 ms

```

[2, 2, 3]
[4, 7, 6]
[7, 8, 12]

```

Транспонирование

```

In [7]: @timed
def matrix_transpose(A):
    n, m = len(A), len(A[0])
    C = [[0.0 for _ in range(n)] for _ in range(m)]

    for i in range(n):
        for j in range(m):
            C[j][i] = A[i][j]
    return C

print_matrix(matrix_transpose(A))

```

matrix_transpose 0.016 ms

```

[1, 4, 7]
[2, 5, 8]
[3, 6, 9]

```

Умножение

```

In [8]: def dot(a, b):
        return sum(x * y for (x, y) in zip(a, b))

@timed
def matrix_mul(A, B):
    # check correctness of dimensions
    n, m = len(A), len(B[0])
    C = [[0.0 for _ in range(n)] for _ in range(m)]

    B_T = matrix_transpose(B)
    for i in range(n):
        for j in range(m):
            C[i][j] = dot(A[i], B_T[j])
    return C

```

```
print_matrix(matrix_mul(A, B))
```

```
matrix_transpose 0.004 ms
```

```
matrix_mul 0.059 ms
```

```
[1, 4, 9]  
[4, 10, 18]  
[7, 16, 27]
```

Библиотека NumPy

- Позволяет работать с многомерными массивами
- Реализует множество базовых матричных операций
- Работает на порядки быстрее самописных реализаций в Python
- Поддерживается большинством сторонних модулей, работающих с матрицами

```
In [9]: # from numpy import * - НЕ ДЕЛАЙТЕ ТАК  
import numpy as np
```

Первый вариант плох по следующим причинам:

- Засорение пространства имён
- Конфликты имён и переопределение встроенных функций
- Проблемы с читаемостью и поддержкой

Текущая версия:

```
In [10]: np.__version__
```

```
Out[10]: '1.26.0'
```

Почему NumPy быстрая

- Значительная часть кода написана на C
- Базовым классом является `ndarray`, имеющий следующие отличия от списков:

1. NumPy array имеет фиксированную длину, задаваемую в момент его создания (списки в Python могут менять размер динамически)
 2. Все элементы в NumPy array имеют один тип
- NumPy array хранится в памяти в виде одного последовательного блока, что позволяет эффективно использовать процессорный кэш и векторные инструкции
 - NumPy можно подключить к высокооптимизированным библиотекам для матричной алгебры (BLAS, LAPACK, Intel MKL)
 - Часть матричных операций может быть распараллелена при наличии в системе нескольких потоков

Способы создания NumPy array

1. Пустой
2. Заполненный нулями
3. Заполненный единицами
4. Заполненный нужным значением

```
In [11]: print(np.empty(shape=[2, 3])) # values are arbitrary
```

```
[[4.68269539e-310  0.00000000e+000  2.14321575e-312]
 [2.22809558e-312  2.46151512e-312  5.68175493e-322]]
```

```
In [12]: print(np.zeros([2, 3]))
```

```
[[0.  0.  0.]
 [0.  0.  0.]]
```

```
In [13]: print(np.ones([2, 3]))
```

```
[[1.  1.  1.]
 [1.  1.  1.]]
```

```
In [14]: print(np.full([2, 3], 3.0))
```

```
[[3.  3.  3.]
 [3.  3.  3.]]
```

Указанные четыре функции имеют аналоги с суффиксом `*_like`, которые создают массив того же размера, что передан в качестве аргумента:

```
In [15]: a = np.zeros([1, 2, 3])
         b = np.ones_like(a)
```

```
print(a)
print(b)
a.shape, b.shape
```

```
[[[0. 0. 0.]
  [0. 0. 0.]]]
[[[1. 1. 1.]
  [1. 1. 1.]]]
```

Out[15]: ((1, 2, 3), (1, 2, 3))

Для создания одномерных массивов можно передавать их длину в виде целого числа:

```
In [16]: np.zeros(5)
```

Out[16]: array([0., 0., 0., 0., 0.])

Для одномерного массива тоже можно писать кортеж:

```
In [17]: np.zeros(shape=(5,))
```

Out[17]: array([0., 0., 0., 0., 0.])

Для больших размерностей, так задавать массив нельзя.

Так неправильно:

```
In [18]: np.zeros(5, 7)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[18], line 1
----> 1 np.zeros(5, 7)

TypeError: Cannot interpret '7' as a data type
```

5. На основе списков

```
In [19]: print(np.array([[1, 2, 3], [4, 5, 6]]))
```

```
[[1 2 3]
 [4 5 6]]
```

Число элементов должно удовлетворять размерности, numpy попытается преобразовать типы:

```
In [20]: print(np.array([[1, 2, 3], [4, 6]]))
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[20], line 1
----> 1 print(np.array([[1, 2, 3], [4, 6]]))
```

ValueError: setting an array element with a sequence. The requested array has an inhomogeneous shape after 1 dimensions. The detected shape was (2,) + inhomogeneous part.

Примечание: Код выше работал до версии 1.24.0. Ранее `np.array([[1, 2, 3], [4, 6]])` автоматически кастилось к `np.array([list([1, 2, 3]), list([4, 6])], dtype=object)`.

```
In [21]: np.array([list([1, 2, 3]), list([4, 6])], dtype=object)
```

```
Out[21]: array([list([1, 2, 3]), list([4, 6])], dtype=object)
```

Тип можно указать явно, и неправильное число элементов приведёт к ошибке даже при версии ниже 1.24.0:

```
In [21]: np.array([[1, 2, 3], [4, 6]], dtype=np.float32)
```

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_2656/4285006876.py in <cell line: 0>()
----> 1 np.array([[1, 2, 3], [4, 6]], dtype=np.float32)
```

ValueError: setting an array element with a sequence. The requested array has an inhomogeneous shape after 1 dimensions. The detected shape was (2,) + inhomogeneous part.

6. Последовательность целых чисел заданной длины

```
In [22]: # Синтаксис аналогичен функции `range`, но в этом случае данные хранятся в г
np.arange(2, 10)
```

```
Out[22]: array([2, 3, 4, 5, 6, 7, 8, 9])
```

7. Разбиение отрезка на равные подотрезки (в исходном и log-масштабе)

```
In [23]: np.linspace(0.0, 1.0, 5), np.logspace(0.0, 1.0, 5)
```

```
Out[23]: (array([0. , 0.25, 0.5 , 0.75, 1.  ]),
          array([ 1.          ,  1.77827941,  3.16227766,  5.62341325, 10.          ]))
```

8. Создание случайных массивов

```
In [154]: np.random.uniform(0, 1, size=(10,)) # равномерное распределение на [0, 1]
```

```
Out[154]: array([0.93674394, 0.98070126, 0.41724372, 0.93178804, 0.31485438,
                 0.74604001, 0.74169372, 0.41523157, 0.6091157 , 0.84881838])
```

```
In [155... np.random.randn(3, 3) # нормальное распределение
```

```
Out[155]: array([[ 0.95340894,  1.42357208, -1.16762005],
 [ 0.15152247, -4.26586472,  0.50680289],
 [ 0.44014878,  0.90981671,  0.73110388]])
```

Чтобы сделать генерацию детерминированной, можно зафиксировать `seed` генерации

```
In [156... np.random.seed(42)
a = np.random.randn(3, 3)

np.random.seed(42)
b = np.random.randn(3, 3)

assert np.all(a == b)
```

Задача 2. Задайте массив `result` размером 50 на 30, состоящий из троек:

```
In [ ]: ### your code here
```

Задача 3. Какая размерность будет у массива, полученного с помощью команды `np.array([[1], [2]])` ?

```
In [ ]: ### your answer here
```

Важные параметры создания ndarray

- `shape` – список или кортеж с размерностями создаваемого массива
- `dtype` – указание типа элементов массива, если массив создаётся на базе объектов из Python, то должно существовать преобразование из типа этих объектов в указываемый тип
- `order` – порядок хранения данных в памяти, по строкам (C-order) или же по столбцам (Fortran-order). По-умолчанию используется C-order

Отличие классов `np.ndarray` и `np.matrix`

- `ndarray` – более общий класс, поддерживающий все возможные операции
- `matrix` – более узкий класс, наследующий `ndarray`. Он поддерживает несколько операций и атрибутов, специфичных для матриц, в удобной нотации

1. В `matrix` знак `*` выполняет матричное умножение, в `ndarray` – поэлементное
 2. Для `matrix` возведение в степень эквивалентно A^n , для `ndarray` это поэлементное возведение.
 3. Класс `matrix` предоставляет удобные свойства-атрибуты для быстрых линейно-алгебраических операций: `.T` – транспонированная, `.I` – обратная, `.H` – сопряжённо-транспонированная матрицы
- При этом все те же операции можно применять и к `ndarray`, если он двумерный и состоит из чисел (начиная с Python 3.5 умножение можно записать как `A @ B`)
 - Рекомендуется пользоваться только `ndarray`, чтобы не вносить путаницу в код и не проверять каждый раз, какого именно типа массив будет обрабатываться

Изменение размерности

`reshape` позволяет изменить размерности массива **без изменения общего числа элементов**:

```
In [24]: a = np.zeros([2, 3, 2])
print(a, a.shape)
```

```
[[[0. 0.]
  [0. 0.]
  [0. 0.]]

 [[0. 0.]
  [0. 0.]
  [0. 0.]]] (2, 3, 2)
```

```
In [25]: b = a.reshape((1, 2 * 3 * 2))
print(b, b.shape, '\n')

b[0][0] = 10 # 'b' is a new view on the same data

print(a, a.shape)
```

```
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.] (1, 12)

 [[[10. 0.]
  [ 0. 0.]
  [ 0. 0.]]

 [[ 0. 0.]
  [ 0. 0.]
  [ 0. 0.]]] (2, 3, 2)
```

Можно проверить, не пересекаются ли в памяти два массива:

```
In [26]: np.may_share_memory(a, b)
```

```
Out[26]: True
```

Отметим, что практически все методы `np.ndarray` доступны в виде функций модуля NumPy:

```
In [27]: c = np.reshape(b, (2 * 3, 2))
print(c, c.shape)
```

```
[[10.  0.]
 [ 0.  0.]
 [ 0.  0.]
 [ 0.  0.]
 [ 0.  0.]
 [ 0.  0.]] (6, 2)
```

Размерность можно изменить и напрямую:

```
In [28]: c.shape = (1, 2 * 3 * 2)
print(c, c.shape)
```

```
[[10.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]] (1, 12)
```

Можно "не заполнять" одну размерность итогового массива, она заполнится автоматически:

```
In [29]: d = c.reshape(2, -1, 3)
print(d, d.shape)
```

```
[[[10.  0.  0.]
 [ 0.  0.  0.]]

 [[ 0.  0.  0.]
 [ 0.  0.  0.]]] (2, 2, 3)
```

Про размерности массивов

```
In [30]: a = np.array([1, 2, 3])
b = np.array([[1, 2, 3]])
c = np.array([[1], [2], [3]])
print(a.shape)
print(b.shape)
print(c.shape)
```

```
(3,)
(1, 3)
(3, 1)
```

- Данные лежат в памяти одним и тем же образом
- Вопрос только во `view` == способ индексации == число и порядок индексов

```
In [31]: print(a.ndim, b.ndim, c.ndim)
```

```
1 2 2
```

Полезная для понимания происходящего статья:

<https://stackoverflow.com/questions/22053050/difference-between-numpy-array-shape-r-1-and-r>

View и копирование

View ссылается на те же данные, но позволяет задать другие размерности массива.

```
In [32]: x = np.arange(10)
v = x.view()
v.shape = (2, 5)
x, v
```

```
Out[32]: (array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]),
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]]))
```

```
In [33]: v[0, 0] = 100
x, v
```

```
Out[33]: (array([100,  1,  2,  3,  4,  5,  6,  7,  8,  9]),
array([[100,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9]]))
```

Если нужно получить копию массива, чтобы не портить переданные данные, пользуйтесь методом `copy` :

```
In [34]: x = np.arange(10)
y = x.copy()
y[:] = 0
x, y
```

```
Out[34]: (array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]), array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0]))
```

Индексирование

- Всё аналогично спискам, можно делать срезы и использовать отрицательные индексы
- Индексы и срезы в многомерных массивах не обязательно разделять квадратными скобками
- Подробное описание: <https://numpy.org/doc/stable/user/basics.indexing.html>

```
In [35]: a = np.array([[1, 2, 3], [3, 4, 5]])

print(a, '\n')
```

```
print(a[0], '\n')
print(a[0, 0: 100500], '\n')
print(a[0][1], '\n')
print(a[-1][-2], '\n')
print(a[-1, -2], '\n')
print(a[0, 0: -1])
```

```
[[1 2 3]
 [3 4 5]]
```

```
[1 2 3]
```

```
[1 2 3]
```

```
2
```

```
4
```

```
4
```

```
[1 2]
```

Важное отличие от питоновских списков: при slicing возвращается **view**, а не копия!
Это позволяет присваивать значения подматрицам.

```
In [36]: a = np.array([[1, 2, 3], [3, 4, 5]])
print(a, '\n')

a[:, 1] = 10
print(a, '\n')
```

```
[[1 2 3]
 [3 4 5]]
```

```
[[ 1 10  3]
 [ 3 10  5]]
```

Логическое индексирование

Индексирование можно производить по логическому массиву такого же размера:

```
In [37]: a = np.arange(10)
i = np.array([j % 2 == 0 for j in range(10)])

print(a, '\n')
print(i, '\n')
print(a[i], '\n')
```

```
[0 1 2 3 4 5 6 7 8 9]
```

```
[ True False  True False  True False  True False  True False]
```

```
[0 2 4 6 8]
```

При этом такие слайсы также позволяют менять исходный массив:

```
In [38]: a[i] = 11  
a
```

```
Out[38]: array([11,  1, 11,  3, 11,  5, 11,  7, 11,  9])
```

С помощью функции `where`, можно находить индексы элементов, заданные маской:

```
In [39]: np.where(a > 4)
```

```
Out[39]: (array([0, 2, 4, 5, 6, 7, 8, 9]),)
```

В случае одного аргумента `np.where` вернет `tuple`. Смотри документацию или [тут](#)

```
In [43]: c = np.arange(12).reshape(3, 2, 2)  
np.where(c > 4)
```

```
Out[43]: (array([1, 1, 1, 2, 2, 2, 2]),  
         array([0, 1, 1, 0, 0, 1, 1]),  
         array([1, 0, 1, 0, 1, 0, 1]))
```

```
In [41]: c
```

```
Out[41]: array([[[ 0,  1],  
                [ 2,  3]],  
               [[ 4,  5],  
                [ 6,  7]],  
               [[ 8,  9],  
                [10, 11]]])
```

```
In [42]: np.where(a > 4, a, 0)
```

```
Out[42]: array([11,  0, 11,  0, 11,  5, 11,  7, 11,  9])
```

Задача 4. Даны два вектора одинаковой длины: `a` и `b`. Оставить в этих векторах только те элементы, которые соответствуют позициям ненулевых элементов в обоих векторах.

```
In [44]: a = np.array([0, 1, 0, 2, 3])  
b = np.array([1, 5, 2, 0, 6])
```

```
In [ ]: ### your code here
```

Проверьте себя:

```
In [ ]: assert a.tolist() == [1, 3]  
assert b.tolist() == [5, 6]
```

```
-----  
AssertionError                                Traceback (most recent call last)  
/tmp/ipython-input-3008322664.py in <cell line: 0>()  
----> 1 assert a.tolist() == [1, 3]  
      2 assert b.tolist() == [5, 6]  
  
AssertionError:
```

Сложное индексирование

По каждой размерности подаются массивы одинаковых размеров, элементы которых соответствуют индексам каждой размерности. Тогда, на выходе будет массив размера этих массивов, элементы которого будут соответствовать элементам исходного массива, взятых в точках, соответствующих поданным массивам.

```
In [45]: X = np.array([[1, 2, 3], [4, 5, 6]])  
X
```

```
Out[45]: array([[1, 2, 3],  
               [4, 5, 6]])
```

```
In [46]: X[[0, 1], [1, 2]], X[[0, 0], [1, 0]]
```

```
Out[46]: (array([2, 6]), array([2, 1]))
```

```
In [47]: Y = np.arange(24).reshape((3, 2, 4))  
Y
```

```
Out[47]: array([[[ 0,  1,  2,  3],  
                [ 4,  5,  6,  7]],  
               [[ 8,  9, 10, 11],  
                [12, 13, 14, 15]],  
               [[16, 17, 18, 19],  
                [20, 21, 22, 23]])
```

```
In [48]: Y[[1, 0], :, [2, 1]]
```

```
Out[48]: array([[10, 14],  
               [ 1,  5]])
```

Сокращенная индексация

Если у вас есть многомерный массив, а вам, например, надо взять определенные индексы только с последней размерности, то можно написать так:

```
In [49]: x = np.arange(80).reshape((2, 2, 4, 5))  
  
print(x[..., 0])  
x[..., 0].shape
```

```
[[[ 0  5 10 15]
   [20 25 30 35]]
```

```
[[40 45 50 55]
 [60 65 70 75]]]
```

Out[49]: (2, 2, 4)

Более хитрый пример:

```
In [50]: x = np.arange(720).reshape((3, 4, 5, 3, 2, 2))
```

```
print(x[2, :, 3, ..., 0])
x[2, :, 3, ..., 0].shape
```

```
[[516 518]
 [520 522]
 [524 526]]
```

```
[[576 578]
 [580 582]
 [584 586]]
```

```
[[636 638]
 [640 642]
 [644 646]]
```

```
[[696 698]
 [700 702]
 [704 706]]]
```

Out[50]: (4, 3, 2)

Арифметические операции

- Арифметические операции в общем случае по-элементные и требуют одинакового размера операндов
- Но часто NumPy может применять их к операндам разного размера с помощью broadcasting, то есть правил обработки операндов разного размера

Примеры операций с массивами одного размера:

```
In [51]: A, B = np.array(A), np.array(B) # were declared previously
```

```
print(A, '\n\n', B, '\n\n', 2 * A, '\n\n', A + B)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

[[1 0 0]
 [0 2 0]
 [0 0 3]]

[[ 2  4  6]
 [ 8 10 12]
 [14 16 18]]

[[ 2  2  3]
 [ 4  7  6]
 [ 7  8 12]]
```

Поэлементные арифметические операции

```
In [52]: np.sin(A)
```

```
Out[52]: array([[ 0.84147098,  0.90929743,  0.14112001],
                [-0.7568025 , -0.95892427, -0.2794155 ],
                [ 0.6569866 ,  0.98935825,  0.41211849]])
```

```
In [53]: np.abs(A)
```

```
Out[53]: array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

```
In [54]: A ** 0.5
```

```
Out[54]: array([[1.          , 1.41421356, 1.73205081],
                [2.          , 2.23606798, 2.44948974],
                [2.64575131, 2.82842712, 3.          ]])
```

Поэлементные операции сравнения:

```
In [55]: A > 3
```

```
Out[55]: array([[False, False, False],
                [ True,  True,  True],
                [ True,  True,  True]])
```

Матричные арифметические операции

```
In [56]: print(A - B)
```

```
[[0 2 3]
 [4 3 6]
 [7 8 6]]
```

```
In [57]: print(A * B)
```

```
[[ 1  0  0]
 [ 0 10  0]
 [ 0  0 27]]
```

```
In [58]: print(B / A)
```

```
[[1.         0.         0.         ]
 [0.         0.4        0.         ]
 [0.         0.         0.33333333]]
```

```
In [59]: print(A @ B)
```

```
[[ 1  4  9]
 [ 4 10 18]
 [ 7 16 27]]
```

Фиктивная размерность

Два способа добавить фиктивную размерность в произвольный массив:

```
In [60]: d[:, np.newaxis].shape
```

```
Out[60]: (2, 1, 2, 3)
```

```
In [61]: d[None, :].shape
```

```
Out[61]: (1, 2, 2, 3)
```

Можно для добавления фиктивной размерности использовать `reshape` :

```
In [63]: x = np.arange(7)
```

```
x, x.reshape(-1, 1)
```

```
Out[63]: (array([0, 1, 2, 3, 4, 5, 6]),
          array([[0],
                 [1],
                 [2],
                 [3],
                 [4],
                 [5],
                 [6]]))
```

```
In [64]: x = np.arange(8).reshape(2, 4)
```

```
x, x.reshape(2, -1, 4)
```

```
Out[64]: (array([[0, 1, 2, 3],
                 [4, 5, 6, 7]]),
          array([[[0, 1, 2, 3]],
                 [[4, 5, 6, 7]]]))
```

Зачем нужно добавлять фиктивную размерность? Например, чтобы проводить различные операции с массивами неравного размера.

Broadcasting

Подробное описание: <http://docs.scipy.org/doc/numpy/user/basics.broadcasting.html>

Пусть нам дана матрица X размером 5×5 и вектор y длины 5. Пусть мы хотим прибавить вектор к каждой строке матрицы.

```
In [65]: x = np.arange(25).reshape(5, 5)
y = np.arange(5)

x, y
```

```
Out[65]: (array([[ 0,  1,  2,  3,  4],
                 [ 5,  6,  7,  8,  9],
                 [10, 11, 12, 13, 14],
                 [15, 16, 17, 18, 19],
                 [20, 21, 22, 23, 24]]),
          array([0, 1, 2, 3, 4]))
```

Наивный способ решения проблемы будет работать правильно!

```
In [66]: x + y
```

```
Out[66]: array([[ 0,  2,  4,  6,  8],
                 [ 5,  7,  9, 11, 13],
                 [10, 12, 14, 16, 18],
                 [15, 17, 19, 21, 23],
                 [20, 22, 24, 26, 28]])
```

Важно: Почему? Правила broadcasting (приведение размеров).

1. Если два массива имеют размерности $(a_1, a_2, \dots, a_n)$ и $(b_1, b_2, \dots, b_n)$ соответственно, то между ними можно проводить поэлементные операции, если для каждого i выполнено одно из трёх условий:
 - $a_i = b_i$
 - $a_i = 1$
 - $b_i = 1$
2. Если поэлементная операция выполняется между массивами разного размера, то к массиву меньшего размера добавляются ведущие фиктивные размерности.

Арифметика и broadcasting

Если по одному из измерений массивы не равны, и у одного из них эта размерность имеет длину 1, то он будет продублирован по этой размерности:

```
In [67]: a = np.array([[1, 2], [3, 4]])
b = np.array([1])
```

```
c = np.array([1, 1])

print(b + c, '\n')
print(a + b, '\n')
print(a + c)
```

```
[2 2]
```

```
[[2 3]
 [4 5]]
```

```
[[2 3]
 [4 5]]
```

Если не совпадает количество размерностей, то массив, у которого их меньше, будет "добиваться" слева размерностями длины 1:

```
In [68]: a = np.ones((2, 3, 4))
b = np.ones(4)

print(a + b) # here a.shape=(2, 3, 4) and b.shape is considered to be (1, 1,
```

```
[[[2. 2. 2. 2.]
  [2. 2. 2. 2.]
  [2. 2. 2. 2.]]
```

```
[[2. 2. 2. 2.]
 [2. 2. 2. 2.]
 [2. 2. 2. 2.]]]
```

Добавим к массиву вектор-строку:

```
In [69]: a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([1, 2, 3])
a + b
```

```
Out[69]: array([[2, 4, 6],
               [5, 7, 9]])
```

Теперь попробуем добавить столбец:

```
In [70]: a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([1, 2])
a + b
```

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_2656/516891486.py in <cell line: 0>()
      1 a = np.array([[1, 2, 3], [4, 5, 6]])
      2 b = np.array([1, 2])
----> 3 a + b

ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

- Ошибка возникла из-за того, что прибавлялась строка с неправильным размером

- Нужно преобразовать строку в столбец (добавить фиктивную размерность), и тогда NumPy поймёт, как с ней работать

Для этого воспользуемся `reshape` (способ хороший, но далеко не единственный):

```
In [71]: a + np.reshape(b, (2, 1))
```

```
Out[71]: array([[2, 3, 4],
               [6, 7, 8]])
```

Скорость и broadcasting

- Рассмотрим на примере на сколько удобнее broadcasting по памяти и времени
- Посчитаем попарные расстояния между векторами (для удобства запишем их в одну таблицу)

```
In [72]: n, d = 2000, 64
X = np.arange(n*d).reshape(n, d) # (n,d)
```

```
In [73]: # Без broadcasting - руками раздуваем до (n,n,d)
@timed
def distances_repeat(X):
    n, d = X.shape
    X1 = np.repeat(X[:, np.newaxis, :], n, axis=1) # (n,n,d)
    X2 = np.repeat(X[np.newaxis, :, :], n, axis=0) # (n,n,d)
    diff = X1 - X2
    return np.sqrt(np.sum(diff*diff, axis=2)) # (n,n)
```

```
In [74]: # C broadcasting - numpy сам "растянет" оси
@timed
def distances_broadcast(X):
    diff = X[:, np.newaxis, :] - X[np.newaxis, :, :] # (n,n,d)
    return np.sqrt(np.sum(diff*diff, axis=2)) # (n,n)
```

```
In [75]: D1 = distances_repeat(X)
D2 = distances_broadcast(X)

print("repeat vs broadcast:", np.allclose(D1, D2))
```

distances_repeat 3743.361 ms

distances_broadcast 1973.283 ms

repeat vs broadcast: True

А теперь вспомним как можно не плодить лишние вычисления)

воспользуемся следующим равенством: $\|x - y\|^2 = \|x\|^2 + \|y\|^2 - 2xy^T$

```
In [76]: @timed
def distances_matmul_no_broadcast(X):
    n, d = X.shape
    XX = np.sum(X * X, axis=1) # (n,)

    XX_row = np.repeat(XX[:, np.newaxis], n, axis=1) # (n,n)
    XX_col = np.repeat(XX[np.newaxis, :], n, axis=0) # (n,n)

    XY = X @ X.T # (n,n)

    D2 = XX_row + XX_col - 2 * XY
    return np.sqrt(D2)
```

```
In [77]: @timed
def distances_matmul(X):
    XX = np.sum(X*X, axis=1)
    D2 = XX[:, np.newaxis] + XX[np.newaxis, :] - 2 * (X @ X.T)
    return np.sqrt(D2)
```

```
In [78]: D3 = distances_matmul_no_broadcast(X)
D4 = distances_matmul(X)

print("repeat vs broadcast:", np.allclose(D3, D4))
```

distances_matmul_no_broadcast 333.507 ms

distances_matmul 266.723 ms

repeat vs broadcast: True

Арифметика и broadcasting

Всегда проверяйте операции с бродкастингом! Легко можно напороться на неприятности

Задача 5. Какие из этих команд будут выполняться с ошибкой?

1. `np.ones((2, 3)) + np.ones(3)`
2. `np.ones(2) + np.ones((2, 3))`
3. `np.zeros((4, 3)) + np.ones((4, 1))`
4. `np.zeros((3, 4)) + np.ones((4, 3))`
5. `np.zeros((1, 3, 5)) + np.zeros((1, 3))`
6. `np.zeros((5, 3, 1)) + np.zeros((1, 5))`

```
In [ ]: ### your answer here
```

Задача 6. Пусть нам дана матрица X размером 10×10 и вектор-столбец y длины 10.

Получите матрицу `result`, полученную прибавлением к каждому столбцу X вектора y (без использования циклов).

```
In [79]: x = np.arange(100).reshape(10, 10)
        y = np.arange(10)
```

```
In [ ]: ## your code here
```

Проверьте себя:

```
In [ ]: assert result[0][0] == 0
        for i in range(1, 10):
            assert result[i][i-1] == 10 * i + (i - 1) * 2 + 1
```

Матричное умножение

Рассмотрим для случая двумерных матриц, кому интересны многомерные, изучайте <https://numpy.org/devdocs/reference/generated/numpy.dot.html>

```
In [80]: print(A @ B, '\n')
        print(A.dot(B), '\n')
        print(np.dot(A, B))
```

```
[[ 1  4  9]
 [ 4 10 18]
 [ 7 16 27]]
```

```
[[ 1  4  9]
 [ 4 10 18]
 [ 7 16 27]]
```

```
[[ 1  4  9]
 [ 4 10 18]
 [ 7 16 27]]
```

Транспонирование

- Рассмотрим для случая двумерных матриц, кому интересны многомерные, изучайте

<https://stackoverflow.com/questions/32034237/how-does-numpys-transpose-method-permute-the-axes-of-an-array>

- При транспонировании (как и `reshape`) возвращается ссылка на те же данные

```
In [81]: print(A, '\n\n', A.T, '\n\n', A.transpose())
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

Стоит быть осторожным с транспонированием:

```
In [82]: # in memory: 0, 1, 2, 3
a = np.arange(4).reshape(2, 2)
b = a.reshape(-1)
c = a.T.reshape(-1)
np.may_share_memory(a, b), np.may_share_memory(a, c)
```

Out[82]: (True, False)

```
In [84]: d = a.T.view()
np.may_share_memory(a, d), d.flags['C_CONTIGUOUS']
```

Out[84]: (True, False)

```
In [85]: print(a.flags['C_CONTIGUOUS'])
print(a.T.flags['C_CONTIGUOUS'])
```

True
False

Атрибут `flags` класса `ndarray` содержит информацию о расположении массива в памяти. `C_CONTIGUOUS` позволяет получить информацию о том, являются ли данные единым последовательным сегментом в C стиле.

Сравнение скорости

Воспользуемся декоратором `timed` для сравнения скорости матричных операций, реализованных в `numpy` и написанных выше с помощью списков:

```
In [86]: @timed
def matrix_add_np(A, B): return A + B

@timed
def matrix_mul_np(A, B): return A @ B
```

```
In [87]: tmp = [range(1000) for _ in range(1000)]
X, Y = np.array(tmp), np.array(tmp)

_ = matrix_add(X, Y)
_ = matrix_add_np(X, Y)
```

```
tmp = [range(200) for _ in range(200)]
X, Y = np.array(tmp), np.array(tmp)

_ = matrix_mul(X, Y)
_ = matrix_mul_np(X, Y)
```

matrix_add 1597.145 ms

matrix_add_np 2.253 ms

matrix_transpose 26.223 ms

matrix_mul 3934.173 ms

matrix_mul_np 12.778 ms

Рассмотрим также следующий пример:

```
In [88]: # суммирование по столбцам
@timed
def matrix_sum_cols(A): return A.sum(axis=0)

# суммирование по строкам
@timed
def matrix_sum_rows(A): return A.sum(axis=1)
```

```
In [95]: n = 100
a = np.arange(n * n).reshape((n, n))

_ = matrix_sum_cols(a)
_ = matrix_sum_rows(a)
```

matrix_sum_cols 0.160 ms

matrix_sum_rows 0.121 ms

Вопрос: Почему суммирование по строкам быстрее, чем суммирование по столбцам?

Ответ:

В памяти значения лежат по строкам (C-order), чтение из памяти последовательных элементов более эффективно. Если бы мы задали хранение по столбцам (Fortran-order), то картина бы изменилась

Агрегирующие функции

Подобных функций много, ищите нужное в документации NumPy

```
In [96]: a = np.array([1, 2, 3, 4, 3, 2, 1, 3, 3])
```

```
print('Min element:      {}'.format(np.min(a)))
print('Min element position: {}'.format(np.argmin(a)))
print('Max element:      {}'.format(np.max(a)))
print('Mean:             {}'.format(np.mean(a)))
print('Sum:              {}'.format(np.sum(a)))
print('Median:           {}'.format(np.median(a)))
print('Cumulative sum:    {}'.format(np.cumsum(a)))
print('Cumulative product: {}'.format(np.cumprod(a)))
```

```
Min element:      1
Min element position: 0
Max element:      4
Mean:             2.4444444444444446
Sum:              22
Median:           3.0
Cumulative sum:    [ 1  3  6 10 13 15 16 19 22]
Cumulative product: [  1  2  6 24 72 144 144 432 1296]
```

Что будет происходить в случае многомерного массива?

В многомерном случае операция применяется к массиву, вытянутому в вектор (flatten):

```
In [97]: a = np.array([[1, 2, 3], [4, 3, 2], [1, 3, 3]])
print(a, '\n')
print(np.max(a))
print(np.cumsum(a))
```

```
[[1 2 3]
 [4 3 2]
 [1 3 3]]

4
[ 1  3  6 10 13 15 16 19 22]
```

Для работы с определёнными размерностями нужно указать явно параметр `axis`:

```
In [98]: print(np.max(a, axis=0), '\n')
print(np.cumsum(a, axis=1))
```

```
[4 3 3]

[[1 3 6]
 [4 7 9]
 [1 4 7]]
```

Как легко запомнить, как работает `axis`:

- после применения функции получится массив, в котором будет отсутствовать размерность указанная в `axis`, но остальные размерности будут иметь такие же значения

Правда иногда всё-таки хочется, чтобы размерность, по которой происходит агрегирование, осталась. В таком случае на помощь приходит параметр `keepdims`

```
In [99]: a = np.array([[1, 2, 3], [4, 3, 2], [1, 3, 3]])
```

```
b = np.max(a, axis=0, keepdims=True)
c = np.max(a, axis=0)

assert np.all(b == c)
print(b.shape, c.shape)
```

(1, 3) (3,)

Задача 7. Дан вектор x и квадратная матрица D . Вычислить вектор значений $y_j = \operatorname{argmin}_i (x_i + D_{ij})$.

```
In [100]: x = np.array([5, 2, 3, 1])
D = np.array([
    [1, 2, 3],
    [2, 2, 4],
    [5, 6, 1],
    [2, 4, 5]
])
```

```
In [ ]: ### your code here
```

Проверьте себя:

```
In [ ]: assert y.tolist() == [3, 1, 2]
```

Конкатенация массивов

Конкатенировать несколько массивов можно с помощью функций `concatenate` (общий случай), `hstack` и `vstack`:

```
In [101]: np.hstack([A, B])

Out[101]: array([[1, 2, 3, 1, 0, 0],
                 [4, 5, 6, 0, 2, 0],
                 [7, 8, 9, 0, 0, 3]])
```

```
In [102]: np.vstack([A, B])

Out[102]: array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9],
                 [1, 0, 0],
                 [0, 2, 0],
                 [0, 0, 3]])
```

Ещё несколько полезных функций

Чтобы построить двумерный массив, состоящий из повторений одномерных, можно использовать `np.tile`:

```
In [103... a = np.array([2, 3, 4])
np.tile(a[np.newaxis, :], (2, 1))
```

```
Out[103]: array([[2, 3, 4],
                [2, 3, 4]])
```

Для того, чтобы повторить каждый элемент в массиве, можно использовать

`np.repeat`

```
In [104... np.repeat([1, 2, 3], repeats=2, axis=0)
```

```
Out[104]: array([1, 1, 2, 2, 3, 3])
```

```
In [105... print(np.all(A == A))
print(np.all(A == B))
print(np.all(A == [1, 2, 3], axis=1))
```

True

False

[True False False]

```
In [106... func = np.vectorize(lambda x: x ** 2)
func(A)
```

```
Out[106]: array([[ 1,  4,  9],
                [16, 25, 36],
                [49, 64, 81]])
```

Для того, чтобы ограничить элементы массива в заданном диапазоне, можно использовать `np.clip`

```
In [107... a = np.arange(9)
print(a)

a = np.clip(a, 2, 7)

print(a)
```

```
[0 1 2 3 4 5 6 7 8]
```

```
[2 2 2 3 4 5 6 7 7]
```

Если хочется поменять некоторые оси, существует функция `np.moveaxis`

```
In [108... a = np.arange(6).reshape(2, 3)
b = np.moveaxis(a, 0, 1)

print(a, b, sep='\n\n')
```

```
[[0 1 2]
 [3 4 5]]
```

```
[[0 3]
 [1 4]
 [2 5]]
```

Для корректной обработки повторяющихся индексов полезна функция `np.`

`<ufunc>.at`

Сравним два похожих куска кода:

```
In [109... a = np.zeros(5, dtype=int)
          idx = [0, 0, 0]
          a[idx] += 1

          a
```

Out[109]: array([1, 0, 0, 0, 0])

```
In [110... a = np.zeros(5, dtype=int)
          np.add.at(a, [0, 0, 0], 1)

          a
```

Out[110]: array([3, 0, 0, 0, 0])

В первом случае индексация происходит через временный буфер, поэтому дубликаты в индексации затирают друг друга. Во втором случае буферизация не происходит и все индексы обрабатываются корректно. Также выполняемые операции являются in-place

Типы и преобразование типов

Атрибут `dtype` хранит информацию о типе массива:

```
In [111... x = np.arange(15)
          y = np.array([1.5, 2.5])

          x.dtype, y.dtype
```

Out[111]: (dtype('int64'), dtype('float64'))

Преобразование типов осуществляется с помощью метода `astype`:

```
In [112... x = x.astype(np.float64)
          x.dtype
```

Out[112]: dtype('float64')

Можно сразу указывать желаемый тип при создании массивов при помощи параметра `dtype`:

```
In [113... np.ones((2, 3), dtype=str)
```

Out[113]: array([['1', '1', '1'],
 ['1', '1', '1']], dtype='<U1')

Если хочется узнать больше информации про тип данных, то можно воспользоваться `np.iinfo` (для целочисленных типов данных) и `np.finfo` (для чисел с плавающей запятой):

```
In [114...] i64 = np.iinfo(np.int64)
            i64.__dict__
```

```
Out[114]: {'dtype': dtype('int64'), 'kind': 'i', 'bits': 64, 'key': 'i64'}
```

```
In [115...] f32 = np.finfo(np.float32)
            f32.__dict__

print('Bits:                {}'.format(f32.bits))
print('Mantissa bits:       {}'.format(f32.nmant))
print('Exponent bits:       {}'.format(f32.nexp))
print('Precision:           {}'.format(f32.resolution))
print('Min value:           {}'.format(f32.min))
print('Max value:           {}'.format(f32.max))
```

```
Bits:                32
Mantissa bits:       23
Exponent bits:       8
Precision:           9.999999974752427e-07
Min value:           -3.4028234663852886e+38
Max value:           3.4028234663852886e+38
```

Небольшой обзор np.einsum

Данная функция выполняет суммирование Эйнштейна над операндами. Правило Эйнштейна: индекс, повторяющийся в выражении дважды, суммируется. Индекс, встречающийся один раз, остаётся в результате. Более подробно можно почитать на [Хабре](#), или [Stack Overflow](#)

Рассмотрим некоторые примеры:

```
In [116...] # Матричное умножение, эквивалент: A @ B
            np.einsum('ij,jk->ik', A, B)
```

```
Out[116]: array([[ 1,  4,  9],
                 [ 4, 10, 18],
                 [ 7, 16, 27]])
```

```
In [117...] # Транспонирование, эквивалент: A.T
            np.einsum('ij->ji', A)
```

```
Out[117]: array([[1, 4, 7],
                 [2, 5, 8],
                 [3, 6, 9]])
```

```
In [118...] # Сумма всех элементов, эквивалент: A.sum()
            np.einsum('ij->', A)
```

```
Out[118]: np.int64(45)
```

```
In [119]: # Получение диагонали матрицы
np.einsum('ii->i', A)
```

```
Out[119]: array([1, 5, 9])
```

Значение символов

- `...` (ellipsis) «все остальные оси» – broadcasting
- `->` разделяет вход и выход (без него – implicit mode)
- `,` разделяет операнды

Примеры задач на матрицы в NumPy

Условие – нельзя использовать

- циклы
- генераторы списков/списковые включения
- map-функции (в т.ч. `np.vectorize`)

Задача 8. Пронумеровать в порядке следования максимальные элементы в векторе.

Для вектора `[1, 2, 3, 3, 2, 1, 3, 1]` должно получиться `[0, 0, 1, 2, 0, 0, 3, 0]`:

```
In [ ]: def task_1(a):
        pass
        ### your code here

print(task_1([1, 2, 3, 3, 2, 1, 3, 1]))
```

```
[0 0 1 2 0 0 3 0]
```

Задача 9. Реализуйте функцию подсчёта произведения ненулевых элементов на диагонали прямоугольной матрицы. Для матрицы `[[1, 0, 1], [2, 0, 2], [3, 0, 3], [4, 4, 4]]` ответом является 3. Если ненулевых элементов нет, функция должна возвращать `None`:

```
In [ ]: def task_2(A):
        pass
        ### your code here

print(task_2(np.array([[1, 0, 1], [2, 0, 2], [3, 0, 3], [4, 4, 4]])))
```

```
3
```

Задача 10. Найти максимальный элемент в вектор-строке среди элементов, перед которым стоит нулевой. Для `[0, 4, 2, 0, 3, 0, 0, 5, 7, 0]` ответ равен 5:

```
In [ ]: def task_3(a):
```

```

pass
### your code here

print(task_3(np.array([0, 4, 2, 0, 3, 0, 0, 5, 7, 0])))

```

5

Задача 11. Дана матрица размерности $N \times K$, N – число объектов, K – число признаков. Подсчитать выборочное среднее и ковариационную матрицу без использования функций `mean` и `cov`.

$$\mathbb{E}[X] = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\text{cov}(X_{(n)}, Y_{(n)}) = \frac{1}{n} \sum_{t=1}^n (X_t - \bar{X}) (Y_t - \bar{Y})$$

```

In [120...] X = np.arange(32).reshape(8, 4)

```

```

In [ ]: ### your code here

```

```

In [ ]: assert mean.tolist() == [14, 15, 16, 17]
assert np.all(cov == 84)

```

Задача 12. Замените все максимальные элементы матрицы A на 0.

```

In [121...] A = np.array([
    [1, 6, 3],
    [6, 2, 1],
    [4, 5, 6],
    [1, 2, 1],
])

```

```

In [ ]: ### your code here

```

```

In [ ]: new_A = np.array([
    [1, 0, 3],
    [0, 2, 1],
    [4, 5, 0],
    [1, 2, 1],
])
assert np.all(A == new_A)

```

Задача 13. Вычислить площадь круга единичного диаметра с центром в (0.5, 0.5) методом Монте-Карло.

```

In [ ]: ### your code here

```

Проверьте себя:

```
In [123... np.pi / 4
```

```
Out[123]: 0.7853981633974483
```

Постройте график, как аппроксимированные значения площади убывают с итерациями:

```
In [124... import matplotlib.pyplot as plt
```

```
### your code here
```

****Задача* 14.**** Даны матрицы A размера $(n \times d)$ и B размера $(m \times d)$. Найти в A все строки, содержащиеся в B , не используя циклы. Оцените сложность полученного алгоритма.

```
In [126... A = np.array([
    [1, 2, 3],
    [1, 2, 1],
    [4, 5, 6],
    [1, 2, 1],
    [4, 2, 1]
])

B = np.array([
    [4, 5, 6],
    [4, 5, 6],
    [1, 2, 3],
    [0, 1, 2]
])
```

```
In [ ]: ### your code here
```

Проверьте себя:

```
In [ ]: assert set(indexes) == {0, 2}
```

****Задача* 15.**** Даны матрицы A размера $(n \times d)$ и B размера $(m \times d)$. Посчитайте матрицу попарных расстояний D размера $(n \times m)$ такую, что $d_{ij} = \{\|a_i - b_j\|\}_{i,j}^{n,m}$, используя broadcasting. Оцените сложность полученного алгоритма. Можно ли существенно её улучшить? Подсказка: раскройте квадрат нормы разности.

```
In [127... m, n, d = 5, 4, 3
```

```
A = np.random.randn(n, d)
B = np.random.randn(m, d)
A, B
```

```
Out[127]: (array([[ 1.89900931, -0.14553644,  0.39363632],
                  [-0.63633925,  0.01400347, -0.76099087],
                  [-1.61890405,  2.42417248, -0.58522202],
                  [ 1.1430863 , -0.25293571, -0.93725198]]),
          array([[ -1.4345951 ,  0.43267296, -0.13733931],
                  [ 0.44477726,  0.68854711, -1.08988488],
                  [-0.14502885, -0.36528957, -0.20137776],
                  [ 0.82980753,  1.1744685 , -0.21105256],
                  [ 0.7741764 , -0.54111047, -0.14222685]]))
```

```
In [ ]: ### your code here
```

Бонус. Несколько полезных библиотек

Введение в библиотеку SciPy

- SciPy – это библиотека для научных вычислений, решения оптимизационных, численных и статистических задач
- Построена поверх NumPy
- Эффективно реализует многие методы линейной алгебры и оптимизации

Полезные ссылки:

- <https://scipy-lectures.org/intro/scipy.html>
- <https://docs.scipy.org/doc/scipy/reference/>

Состав SciPy

Что кратко рассмотрим:

- `linalg` – линейная алгебра
- `optimize` – методы оптимизации и решения уравнений
- `sparse` – разреженные матрицы
- `stats` – вероятностные распределения и статистика

Состав SciPy

Что есть ещё:

- `cluster` – алгоритмы кластеризации
- `constants` – физические и математические константы
- `fftpack` – инструменты для быстрого преобразования Фурье

- `integrate` – интегрирование и решение ОДУ
- `interpolate` – интерполяция и сплайны
- `io` – библиотека для работы с разными форматами входных и выходных данных
- `ndimage` – обработка N-мерных изображений
- `odr` – реализация Orthogonal distance regression
- `signal` – обработка сигналов
- `spatial` – пространственные структуры данных и алгоритмы
- `special` – набор специальных полезных функций
- `misc` – набор технических инструментов

Модуль `scipy.linalg`

```
In [128... import numpy as np
from scipy import linalg

A = np.array([[1,2], [3, 4]])
```

```
In [129... linalg.det(A) # determinant
```

```
Out[129]: np.float64(-2.0)
```

```
In [130... linalg.inv(A) # inverted matrix
```

```
Out[130]: array([[ -2. ,  1. ],
                [ 1.5, -0.5]])
```

```
In [131... linalg.norm(A) # matrix and vector norms
```

```
Out[131]: np.float64(5.477225575051661)
```

Модуль `scipy.linalg`

```
In [132... for e in linalg.eig(A): # eigen values and vectors
    print(e)
```

```
[-0.37228132+0.j  5.37228132+0.j]
[[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

```
In [133... U, diags, V_T = linalg.svd(A) # SVD
U @ np.diag(diags) @ V_T
```

```
Out[133]: array([[1., 2.],
                [3., 4.]])
```

```
In [134... P, L, U = linalg.lu(A) # pivoted LU decomposition
P @ L @ U
```

```
Out[134]: array([[1., 2.],
                [3., 4.]])
```

Модуль scipy.optimize

Сгенерируем точки с помощью синусоиды с шумом и попробуем восстановить параметры функции

```
In [135... from scipy import optimize
```

```
In [136... x_data = np.linspace(-5, 5, num=50)
y_data = 2.9 * np.sin(1.5 * x_data) + np.random.normal(size=50)
```

```
In [137... def func_to_fit(x, a, b):
    return a * np.sin(b * x)
```

```
In [138... params, params_covariance = optimize.curve_fit(func_to_fit, x_data, y_data)
print(params)
```

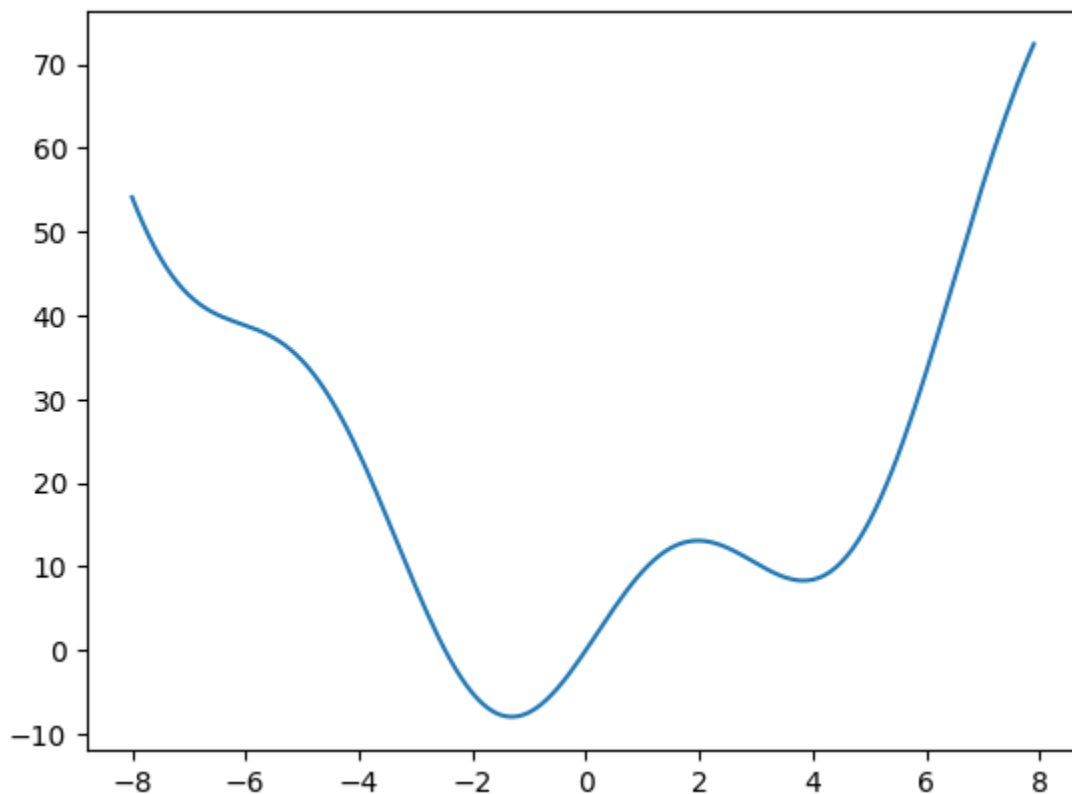
```
[2.85710204 1.53345402]
```

Модуль scipy.optimize

Найдём минимум скалярной функции на отрезке (зависит от стартового приближения)

```
In [140... def f(x):
    return x**2 + 10 * np.sin(x)

x = np.arange(-8, 8, 0.1)
plt.plot(x, f(x))
plt.show()
```



Вопрос: В чём проблема данного графика?

Ответ:

Перечеслим некоторые недочёты: отсутствие сетки, отсутствие заголовка, график отрисован не в векторном формате и т.п.

Некоторые элементы хорошего графика:

Важно: Графики должны быть с одной стороны понятными и информативными, а с другой стороны *красивыми*. Вот несколько пунктов, которые помогут удовлетворить этим требованиям:

1. Все графики должны быть отрисованы в векторном формате. Обратите внимание, что смена режима графиков с динамического на статический и обратно может приводить к сбросу параметров отрисовки графиков. Переход в векторный режим можно выполнить с помощью команды

```
matplotlib_inline.backend_inline.set_matplotlib_formats('pdf', 'svg')
```
2. На всех графиках без исключения должна быть отрисована сетка
3. Все графики и группы графиков должны иметь заголовок (`title`)
4. При необходимости оси должны быть подписаны
5. Если на графике отображено несколько сущностей (линии/точки/bar разных цветов, формы и так далее), то необходима исчерпывающая легенда
6. Все линии на графиках должны быть чётко видны (нет похожих цветов или цветов, сливающихся с фоном и так далее)
7. Масштаб по каждой оси на графике должен быть выбран правильно. Используйте масштабы `log`, `symlog` по необходимости
8. Если отображена величина, имеющая очевидный диапазон значений (например, проценты могут быть от 0 до 100), то желательно масштабировать ось на весь диапазон значений (исключением является случай, когда вам необходимо показать малое отличие, которое незаметно в таких масштабах)
9. Частота отметок по каждой оси должна быть тщательно подобрана, по необходимости задавайте `[xy]ticks`, `[xy]ticklabels` вручную. Подписи тиков на осях не должны сливаться как на одной оси, так и между ними
10. Помните, что matplotlib умеет выполнять [рендеринг LaTeX](#). Используйте эту возможность для написания формул в заголовках, легенде и в подписях осей
11. Используйте *красивую* цветовую палитру с хорошо различимыми цветами. Примеры цветовых палитр можно посмотреть [здесь](#)
12. Графики должны быть не супер-микро и не супер-макро по размерам, так, чтобы можно было увидеть все, что нужно

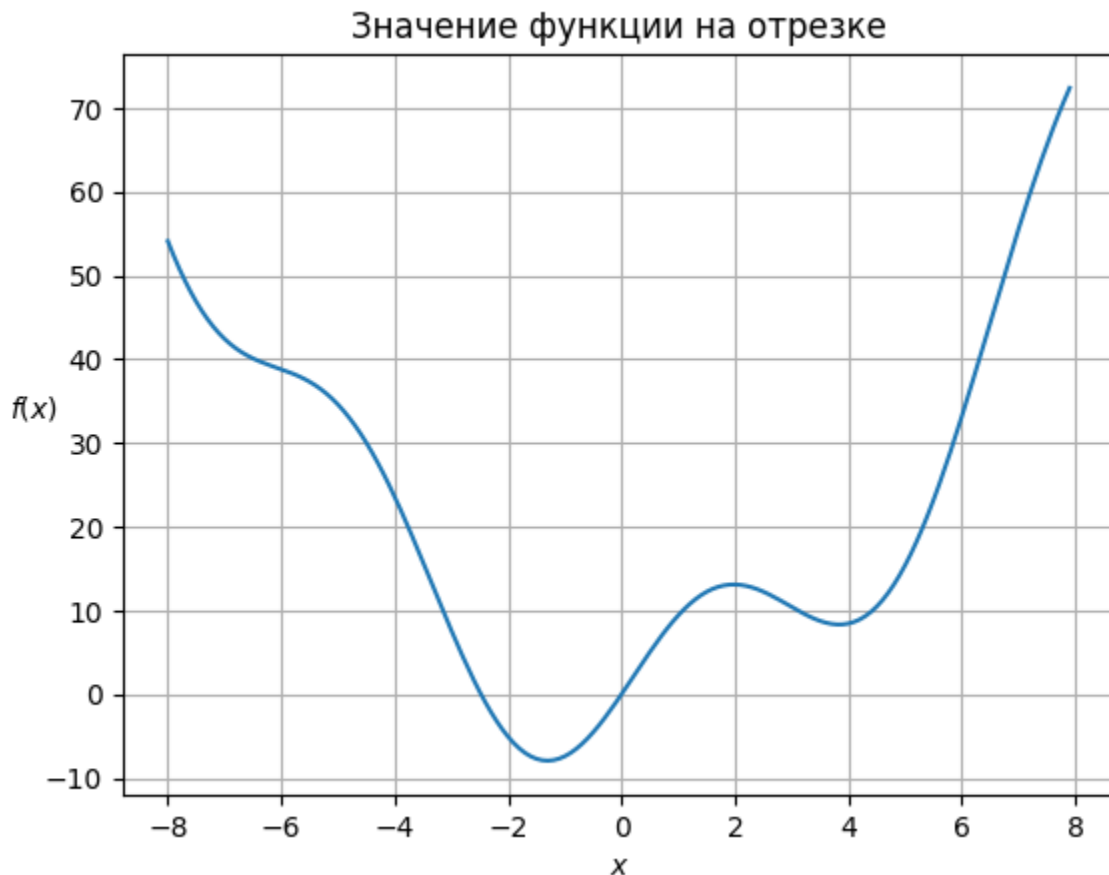
Этот список можно на [GitHub репозитории курса](#).

Задание. Нарисуйте график, который удовлетворял бы элементам хорошего графика

```
In [150]: x = np.arange(-8, 8, 0.1)
plt.plot(x, f(x))

plt.title('Значение функции на отрезке')
plt.xlabel('$x$')
plt.ylabel('$f(x)$', rotation=0)
plt.grid()

plt.show()
```



```
In [ ]: result = optimize.minimize(f, x0=4, bounds=((-8, 8)),) # 'bounds' is optional
result.x
```

```
Out[ ]: array([3.83746782])
```

Найдём минимум скалярной функции на отрезке (зависит от стартового приближения)

```
In [ ]: result = optimize.minimize(f, x0=0, bounds=((-8, 8)),) # 'bounds' is optional
result.x
```

```
Out[ ]: array([-1.30643988])
```

```
In [ ]: result = optimize.basinhopping(f, x0=4, niter=1000)
result.x
```

```
Out[ ]: array([-1.30644001])
```

Функция `optimize.minimize` имеет параметр `method`, в котором можно определять конкретный метод оптимизации, подходящий для решаемой задачи

Найдём нули скалярной функции

```
In [ ]: root = optimize.root(f, x0=1)
        root.x
```

```
Out[ ]: array([0.])
```

```
In [ ]: root2 = optimize.root(f, x0=-2)
        root2.x
```

```
Out[ ]: array([-2.47948183])
```

Находим только один корень, ближайший к стартовой точке, нужно идти по сетке для поиска нескольких

Модуль `scipy.sparse`

- Включает в себя набор различных типов разреженных матриц и инструментов для работы с ними
- Мы рассмотрим основные форматы разреженных матриц, которые могут быть полезны в приложениях

- **Coordinate Format (COO)**

- представляет собой 3 массива `row`, `col`, `data`
- `row` содержит номера строк ненулевых элементов
- `col` содержит номера столбцов ненулевых элементов
- `data` содержит сами ненулевые элементы

```
In [ ]: from scipy.sparse import coo_matrix

S = np.array([[1, 0, 0], [4, 0, 6], [0, 8, 0]])

cm = coo_matrix(S)
cm.row, cm.col, cm.data
```

```
Out[ ]: (array([0, 1, 1, 2], dtype=int32),
        array([0, 0, 2, 1], dtype=int32),
        array([1, 4, 6, 8]))
```

- **Compressed Sparse Row Format (CSR)**

- Представляет собой три массива: `indices`, `indptr`, `data`
- `indices` содержит индексы столбцов ненулевых элементов

- `data` содержит значения ненулевых элементов
- `indptr` содержит старты строк в `indices` и `data` по следующим правилам:
 - длина `indptr` равна числу строк + 1, последний элемент равен числу ненулевых элементов
 - ненулевые значения i -й строки лежат в `data[indptr[i]: indptr[i + 1]]`
 - их индексы столбцов – в `indices[indptr[i]: indptr[i + 1]]`
 - элемент (i, j) доступ в `data[indptr[i]+k]`, где `k` – это позиция `j` в `indices[indptr[i]: indptr[i + 1]]`

```
In [ ]: from scipy.sparse import csr_matrix

S = np.array([[1, 0, 0], [4, 0, 6], [0, 8, 0]])

csrm = csr_matrix(S)
csrm.indices, csrm.indptr, csrm.data
```

```
Out[ ]: (array([0, 0, 2, 1], dtype=int32),
         array([0, 1, 3, 4], dtype=int32),
         array([1, 4, 6, 8], dtype=int32))
```

• Compressed Sparse Column Format (CSC)

- Представляет собой три массива: `indices`, `indptr`, `data`
- `indices` содержит индексы строк ненулевых элементов
- `data` содержит значения ненулевых элементов
- `indptr` содержит старты столбцов в `indices` и `data` по следующим правилам:
 - длина `indptr` равна числу столбцов + 1, последний элемент равен числу ненулевых элементов
 - ненулевые значения i -го столбца лежат в `data[indptr[i]: indptr[i + 1]]`
 - их индексы строк – в `indices[indptr[i]: indptr[i + 1]]`
 - элемент (i, j) доступ в `data[indptr[j]+k]`, где `k` – это позиция `i` в `indices[indptr[j]: indptr[j + 1]]`

```
In [ ]: from scipy.sparse import csc_matrix

S = np.array([[1, 0, 0], [4, 0, 6], [0, 8, 0]])

cscm = csc_matrix(S)
cscm.indices, cscm.indptr, cscm.data
```

```
Out[ ]: (array([0, 1, 2, 1], dtype=int32),
        array([0, 2, 3, 4], dtype=int32),
        array([1, 4, 8, 6], dtype=int32))
```

Модуль scipy.stats

По выборке восстанавливаем параметры известного распределения

```
In [ ]: from scipy import stats

samples = np.random.normal(size=1000, loc=9.8, scale=3.5)
loc, scale = stats.norm.fit(samples)

print(loc, scale)
```

```
9.805095456995904 3.5348333765014224
```

```
In [ ]: samples = np.random.beta(a=1, b=4.6, size=5000) + 10

a, b, loc, scale = stats.beta.fit(samples)
print(a, b, loc, scale)
```

```
0.9841012366717884 4.212599522117305 10.000024072169941 0.9273459800553958
```

Посчитаем статистики выборки:

```
In [ ]: samples = np.random.normal(size=1000, loc=0.0, scale=1.0)

np.mean(samples)
```

```
Out[ ]: -0.01412943515822246
```

```
In [ ]: np.median(samples)
```

```
Out[ ]: -0.02531545160897348
```

```
In [ ]: stats.scoreatpercentile(samples, 50) # median
```

```
Out[ ]: -0.02531545160897348
```

```
In [ ]: stats.scoreatpercentile(samples, 90)
```

```
Out[ ]: 1.2743559538218574
```

Проверка статистических гипотез

- Общая идея:
 - Дана наблюдаемая выборка данных
 - Формулируется гипотеза H_0 – определённое предположение о распределении вероятностей, лежащем в основе наблюдаемой выборки

- Формулируется альтернативная гипотеза H_1
- Выбирается *статистический критерий* (обычно это какая-то статистика выборки)
- На основании выборки вычисляется значение статистического критерия T
- На основании этого значения вычисляется *достигаемый уровень значимости* $p(T)$ (p-value) – вероятность получить значение критерия такое же или ещё более экстремальное, чем T , при условии справедливости H_0
- Если $p(T)$ достаточно мало (например, меньше уровня значимости α), то H_0 отвергается
- Проверка гипотез доступна в `scipy.stats` и основанном на нем модуле `statsmodel`

Пример: Т-критерий

- Т-критерий (тест Стъдента) используется для проверки гипотез о
 - равенстве матожиданий двух выборок
 - равенстве матожидания одной выборки заданному значению
- В случае двух выборок, которые должны быть из распределения, близкого к нормальному, считается величина

$$t = \frac{m_1 - m_2}{\sqrt{\frac{d_1}{n_1} - \frac{d_2}{n_2}}},$$

где - m_i – выборочные средние - d_i – выборочные дисперсии - n_i – размеры выборок

- Определим функцию для генерации и визуализации выборок из нормального распределения:

```
In [ ]: def gen_normal_samples(mean_1, mean_2):
    samples1 = np.random.normal(mean_1, size=1000)
    samples2 = np.random.normal(mean_2, size=1000)

    bins = np.linspace(-4, 4, 30)
    histogram1, bins = np.histogram(samples1, bins=bins, density=True)
    histogram2, bins = np.histogram(samples2, bins=bins, density=True)

    plt.figure(figsize=(6, 4))
    plt.hist(samples1, bins=bins, density=True, label="Samples 1")
    plt.hist(samples2, bins=bins, density=True, label="Samples 2")
    plt.legend(loc='best')
    plt.show()

    return samples1, samples2
```

- Воспользуемся тестом `scipy.stats.ttest_ind` для проверки гипотезы о равенстве средних выборок:

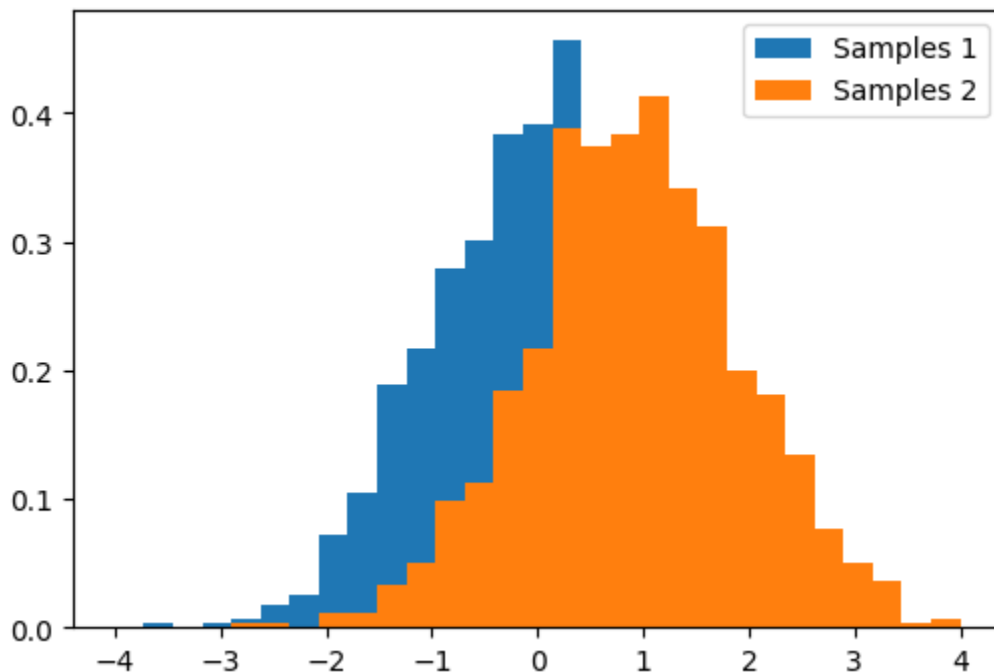
```
In [ ]: stats.ttest_ind

#Calculate the T-test for the means of *two independent* samples of scores.

#This is a two-sided test for the null hypothesis that 2 independent samples
#have identical average (expected) values. This test assumes that the
#populations have identical variances by default.
```

- Посчитаем статистический Т-тест для случая выборок из нормального распределения с априорно далекими средними:

```
In [ ]: samples_1, samples_2 = gen_normal_samples(mean_1=0, mean_2=1)
```

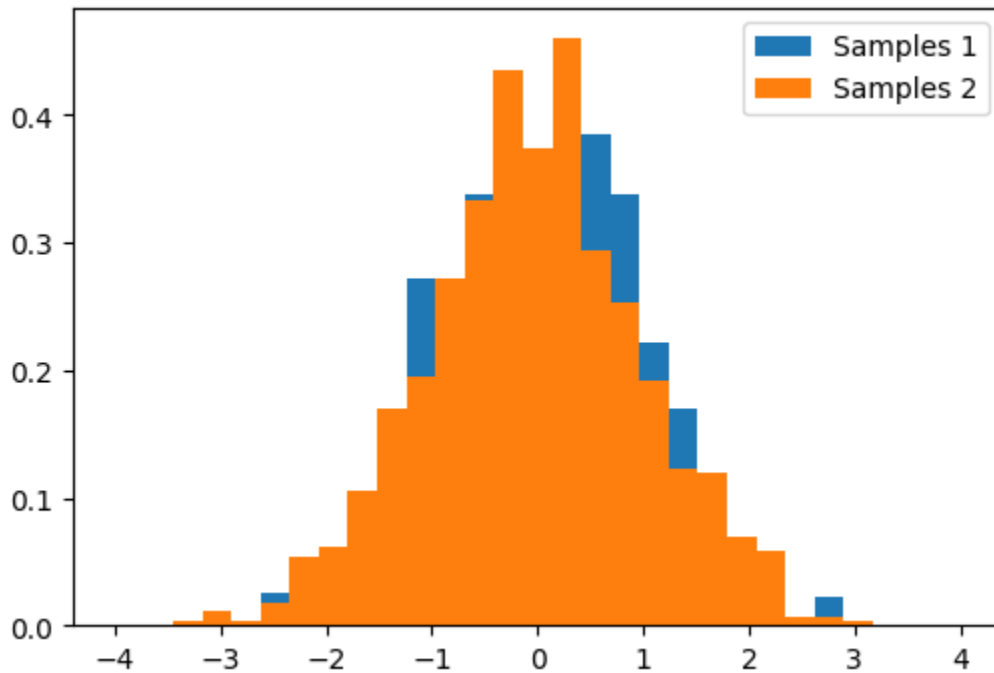


```
In [ ]: t_score, p_value = stats.ttest_ind(samples_1, samples_2)
        t_score, p_value
```

```
Out[ ]: (-21.046576674343378, 5.5371372236298646e-89)
```

- Большое по модулю значений `t_score` означает большую разность между двумя порождающими случайными процессами
- Значение `p_value` характеризует вероятность того, что обе выборки были порождены процессами с одинаковым матожиданием
- Посчитаем статистический Т-тест для случая выборок из нормального распределения с априорно близкими средними:

```
In [ ]: samples_1, samples_2 = gen_normal_samples(mean_1=-0.005, mean_2=0.005)
```



```
In [ ]: t_score, p_value = stats.ttest_ind(samples_1, samples_2)
t_score, p_value
```

```
Out[ ]: (1.1447576071860142, 0.25244669268365644)
```

Библиотека NetworkX

- Позволяет работать с графами, в узлах которых могут находиться любые объекты
- Имеет встроенные процедуры создания графов базовых типов
- Реализует множество алгоритмов на графах, позволяет получать характеристики графов
- Поддерживаются основные форматы хранения графов в файлах
- Имеет встроенные инструменты для визуализации графов (на базе matplotlib и GraphViz)
- В качестве базовой структуры данных использует словарь (`dict`)

Основные классы графов

- `Graph` – простой неориентированный граф. Кратные ребра игнорируются, допускаются петли.
- `DiGraph` – ориентированная версия `Graph`
- `MultiGraph` – неориентированный граф, допускающий кратные ребра
- `MultiDiGraph` – ориентированный граф, допускающий кратные ребра

```
In [ ]: import networkx as nx
G = nx.Graph()
G
```

```
Out[ ]: <networkx.classes.graph.Graph at 0x18ac56e0fa0>
```

Создание графа

Доступны три способа создания:

- Явное добавление вершин и ребер в скрипте
- Генерация графа с помощью алгоритмов создания стандартных сетевых топологий
- Импорт существующего графа из файла

Явное создание графа:

```
In [ ]: G.add_node(1)
G.add_nodes_from([2, 3])
G.nodes
```

```
Out[ ]: NodeView((1, 2, 3))
```

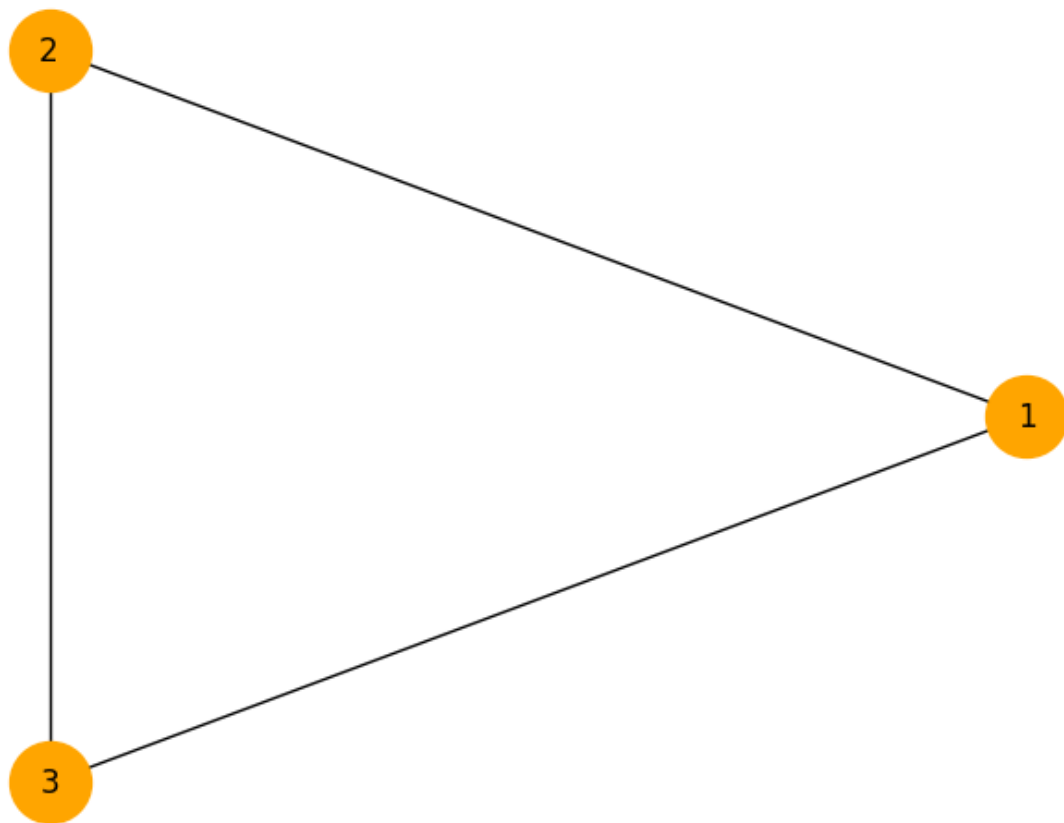
```
In [ ]: G.add_edge(1, 2)
G.add_edges_from([(2, 3), (1, 3)])
G.edges
```

```
Out[ ]: EdgeView([(1, 2), (1, 3), (2, 3)])
```

Визуализация графа

Построенный граф можно несложно визуализировать на плоскости средствами NetworkX:

```
In [ ]: nx.draw_circular(G, node_color='orange', node_size=1000, with_labels=True)
```



Удаление данных из графа

Можно удалить ребра:

```
In [ ]: G.remove_edge(1, 3)
G.edges
```

```
Out[ ]: EdgeView([(1, 2), (2, 3)])
```

Можно удалить вершину (будут удалены все инцидентные ребра):

```
In [ ]: G.remove_node(2)
G.edges
```

```
Out[ ]: EdgeView([])
```

Можно очистить весь граф:

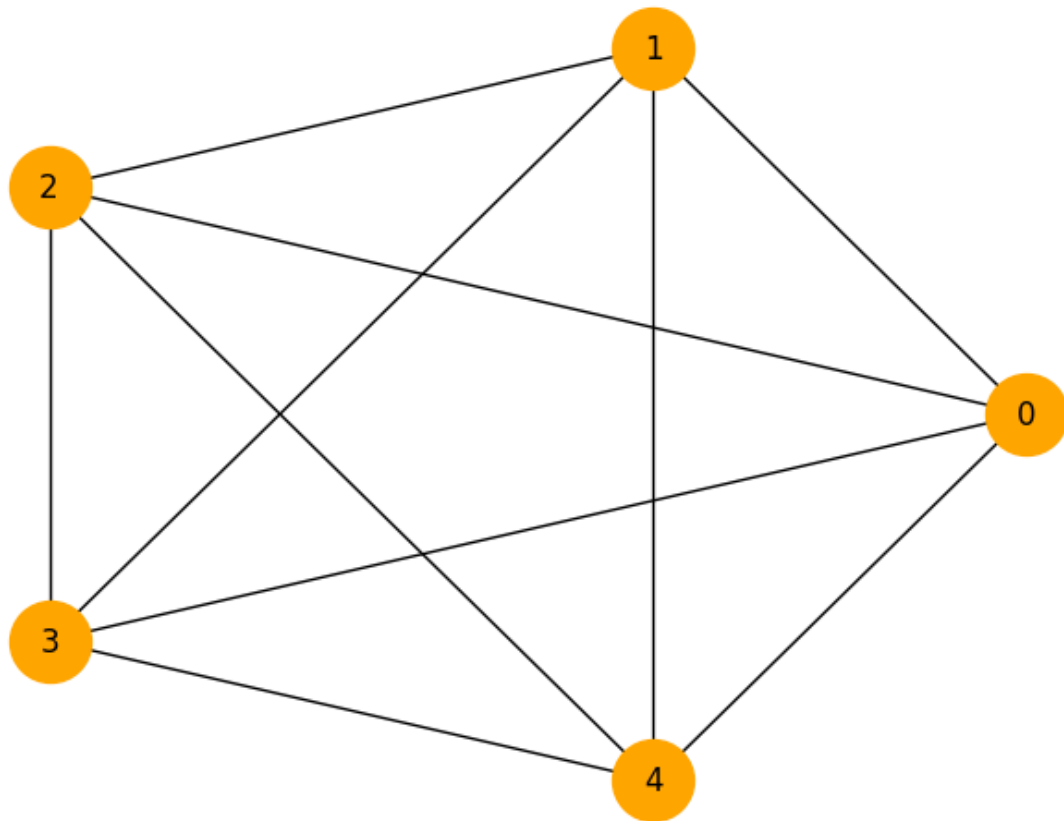
```
In [ ]: G.clear()
print(G.number_of_edges(), G.number_of_nodes())
```

```
0 0
```

Генерация графов

Известные стандартные графы можно создавать с помощью встроенных методов:

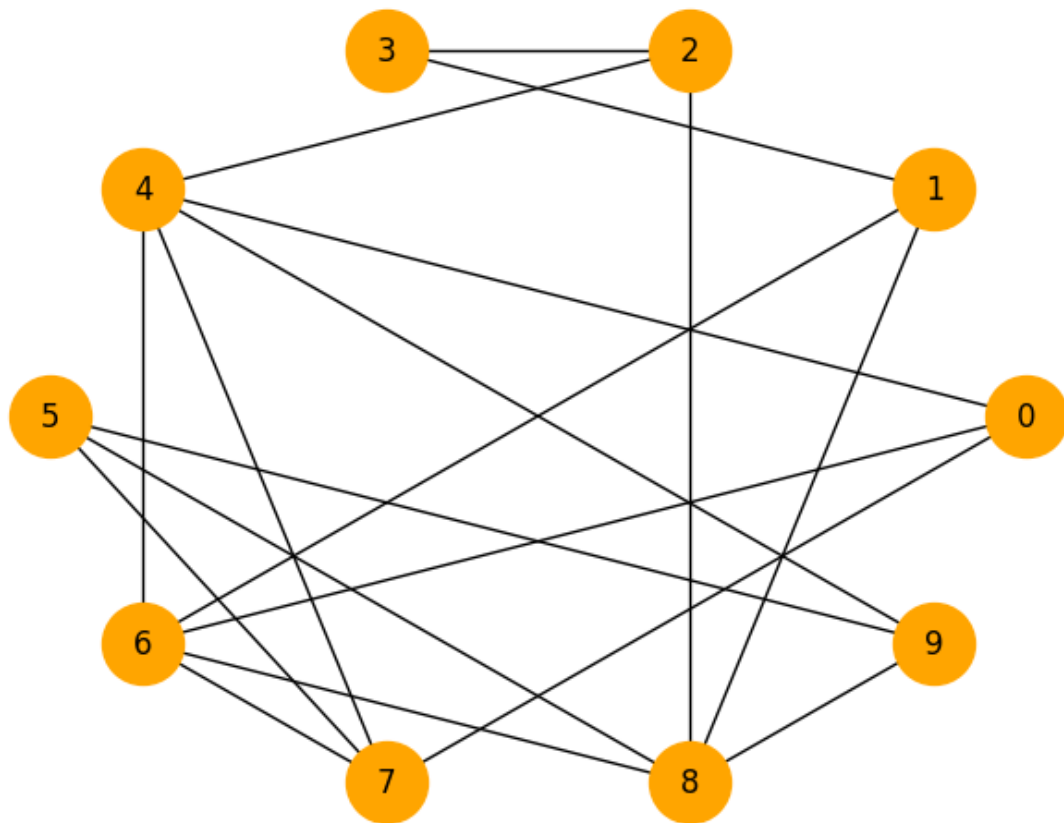
```
In [ ]: K_5 = nx.complete_graph(5)
        nx.draw_circular(K_5, node_color='orange', node_size=1000, with_labels=True)
```



Генерация графов

Генерировать также можно случайные графы:

```
In [ ]: er = nx.erdos_renyi_graph(n=10, p=0.4)
        nx.draw_circular(er, node_color='orange', node_size=1000, with_labels=True)
```



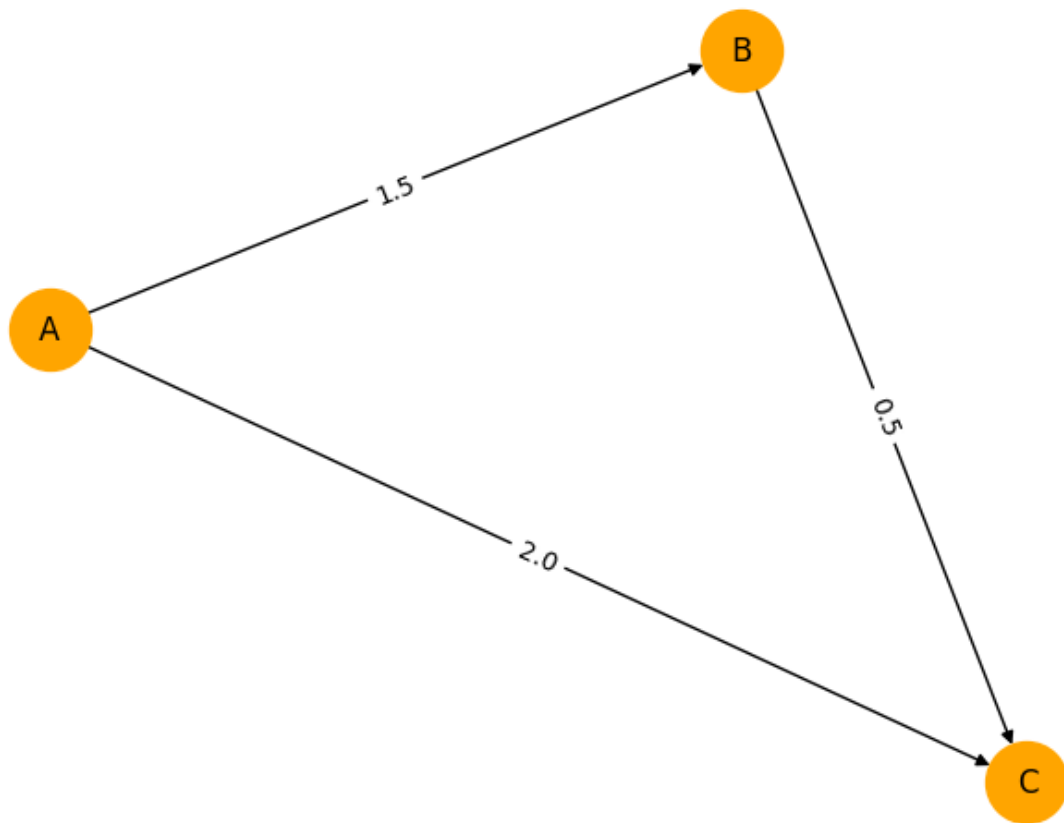
Создание графа из словаря или списка

```
In [ ]: g = {'A': {'B': {'weight': 1.5}, 'C': {'weight': 2.0}},
            'B': {'C': {'weight': 0.5}}}

G = nx.DiGraph(g)

pos = nx.spring_layout(G, seed=7)
nx.draw(G, pos, node_color='orange', node_size=1000, with_labels=True)
labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
```

```
Out[ ]: {('A', 'B'): Text(-0.14333660609105836, 0.4999999999999999, '1.5'),
         ('A', 'C'): Text(-0.04605598989461876, -0.4263681303277567, '2.0'),
         ('B', 'C'): Text(0.1893925959856784, -0.07363186967224339, '0.5')}
```



Конвертация графа в словарь или список

Конвертация в словарь словарей сохраняет всю информацию:

```
In [ ]: nx.convert.to_dict_of_dicts(G)
```

```
Out[ ]: {'A': {'B': {'weight': 1.5}, 'C': {'weight': 2.0}},
         'B': {'C': {'weight': 0.5}},
         'C': {}}
```

Конвертация в список – только информацию о структуре графа:

```
In [ ]: nx.convert.to_dict_of_lists(G)
```

```
Out[ ]: {'A': ['B', 'C'], 'B': ['C'], 'C': []}
```

Операции над графами

<code>nx.subgraph(G, nodes)</code>	<i># induced subgraph view of G on nodes in nodes</i>
<code>nx.union(G1, G2)</code>	<i># graph union</i>
<code>nx.disjoint_union(G1, G2)</code>	<i># graph union assuming all nodes are different</i>
<code>nx.cartesian_product(G1, G2)</code>	<i># return Cartesian product graph</i>
<code>nx.compose(G1, G2)</code>	<i># combine graphs identifying nodes</i>

```

common to both
nx.complement(G)           # graph complement
nx.create_empty_copy(G)    # return an empty copy of the same
                             graph class
nx.to_undirected(G)        # return an undirected representation
                             of G
nx.to_directed(G)          # return a directed representation of
                             G

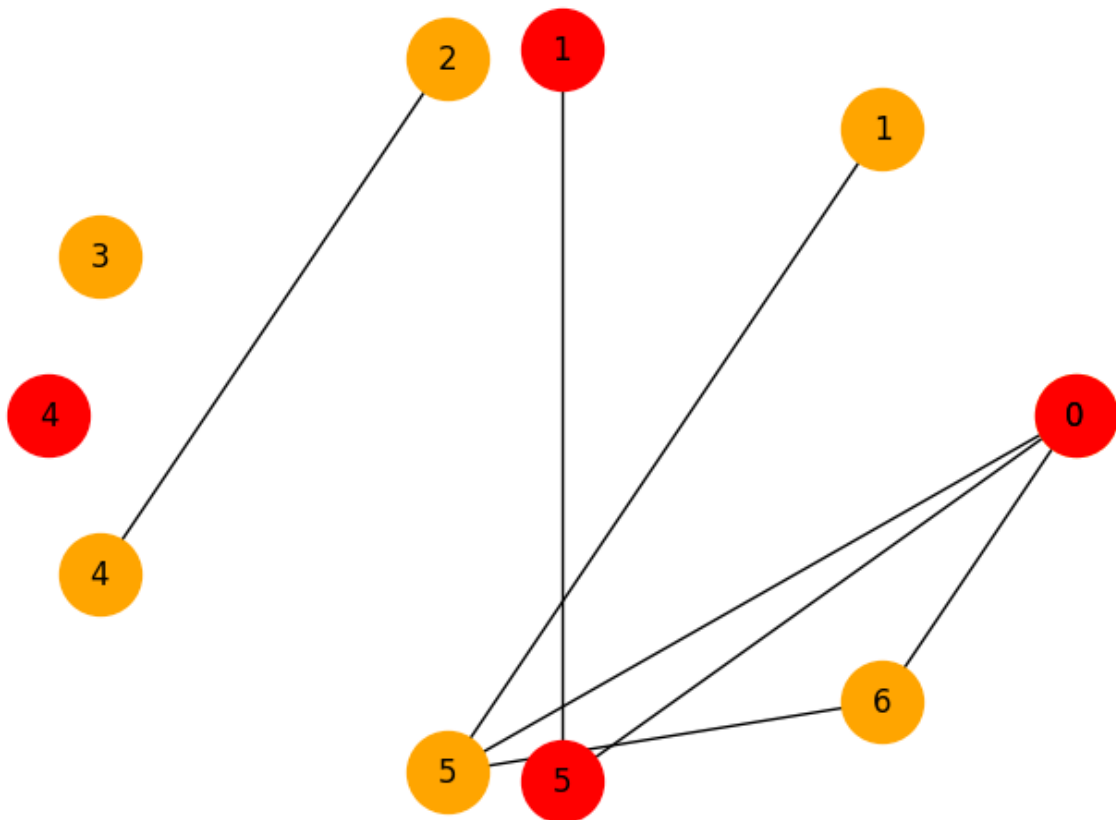
```

Выделение подграфа:

```

In [ ]: G = nx.erdos_renyi_graph(n=7, p=0.4)
nx.draw_circular(G, node_color='orange', node_size=1000, with_labels=True)
nx.draw_circular(nx.subgraph(G, [0, 1, 4, 5]), node_color='red', node_size=1

```



Алгоритмы на графах

В NetworkX реализовано много полезных алгоритмов для анализа и кластеризации графов, полный список есть в [документации](#)

Поиск кратчайших путей

Найдем кратчайший путь между парой вершин:

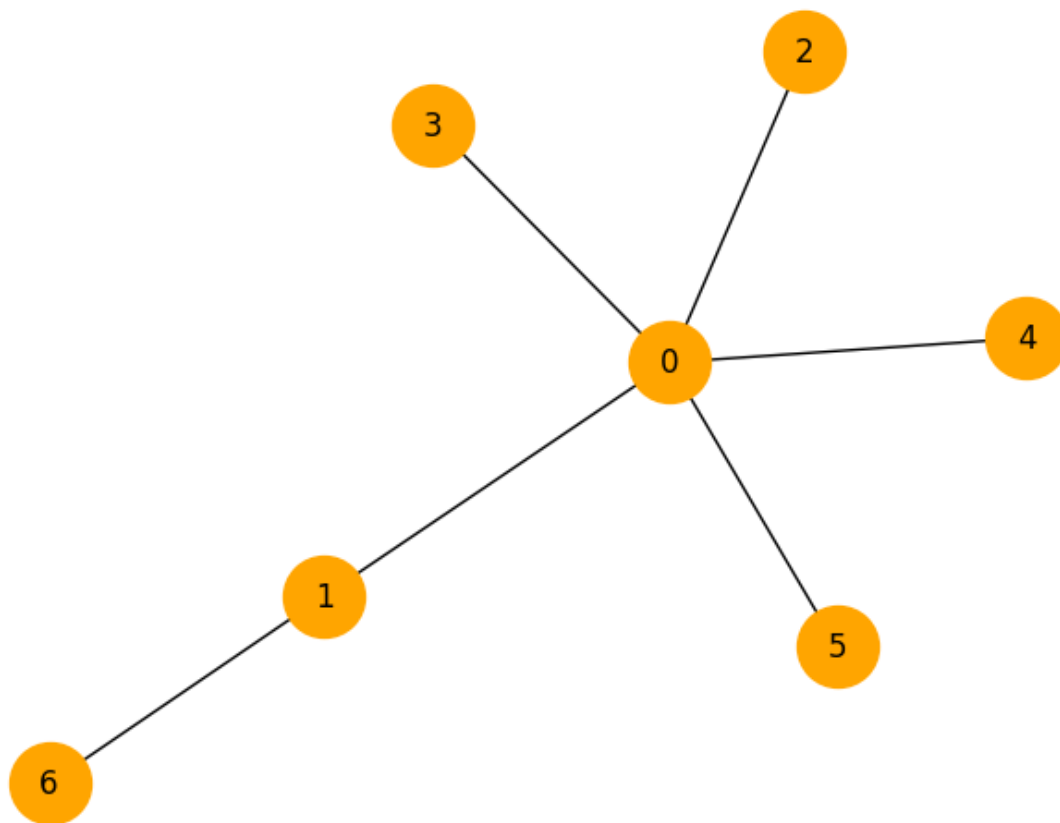
```

In [ ]: G = nx.full_rary_tree(5, 7)

```

```
nx.draw(G, node_color='orange', node_size=1000, with_labels=True)  
print(nx.shortest_path(G, source=6, target=3))
```

```
[6, 1, 0, 3]
```

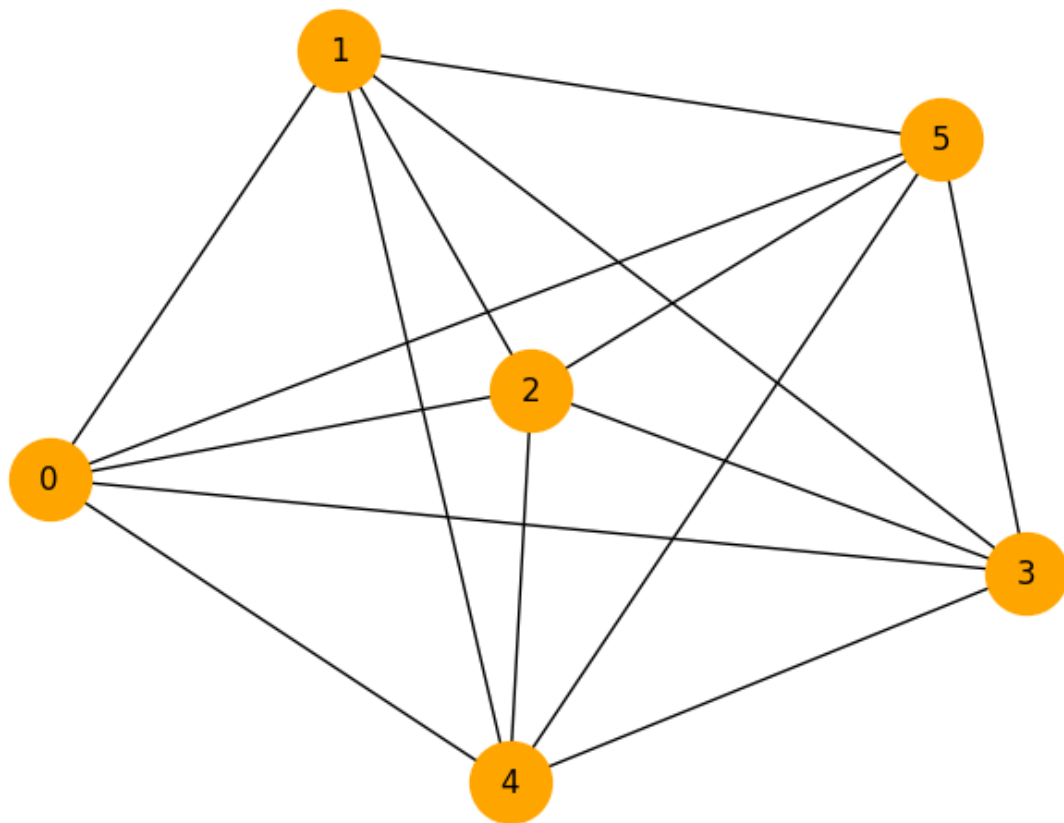


Поиск клик

Найдем все клики (полные подграфы) в алгоритме:

```
In [ ]: G = nx.complete_graph(6)  
nx.draw(G, node_color='orange', node_size=1000, with_labels=True)  
[e for e in nx.algorithms.clique.find_cliques(G)]
```

```
Out[ ]: [[0, 1, 2, 3, 4, 5]]
```



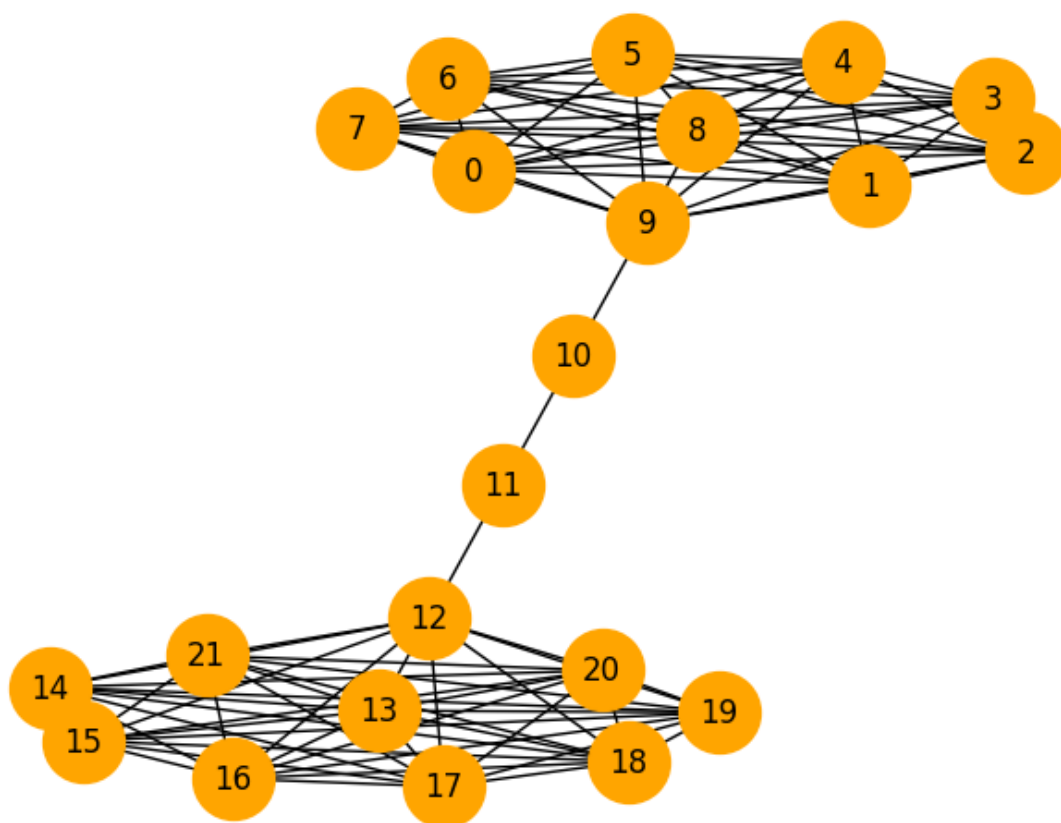
Максимизация модулярности

Выделение сообществ методом жадной максимизации модулярности:

```
In [ ]: G = nx.barbell_graph(10, 2)

nx.draw_kamada_kawai(G, node_color='orange', node_size=1000, with_labels=True)
print(list(nx.algorithms.community.greedy_modularity_communities(G)))

[frozenset({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}), frozenset({12, 13, 14, 15, 16, 17, 18, 19, 20, 21})]
```



По факту кластеризация и выделение сообществ – одна и та же задача, но в алгоритмах (и в реализациях) их часто описывают отдельно

Спасибо за внимание!